



OPEN

Generating detectors from anomaly samples via negative selection for network intrusion detection

Zhiyong Li[✉], Xiang Wei, Chunyan Li & Jianhong Sun

Negative selection algorithms (NSAs), which simulate the human immune mechanism for network anomaly detection, represent a promising approach in this field. Traditional NSAs randomly generate candidate detectors in high-dimensional feature subspaces. However, network anomaly detection datasets such as NSL-KDD typically exhibit sample distributions concentrated in low-dimensional subspaces. This dimensional mismatch causes traditional NSAs to underperform due to insufficient mature detector generation. To address this limitation, we leverage underutilized anomaly samples from the training set as candidate detector centers. Since these samples encode critical information about feature space distribution, they enable effective generation of mature detectors within the relevant low-dimensional subspaces. Furthermore, to mitigate misclassification of anomalies within coverage holes, we implement secondary classification based on the class attributes of nearest neighbor samples. Experiments on the NSL-KDD and UNSW-NB15 datasets showed our method outperforming eight other algorithms.

Keywords Negative Selection Algorithm, Artificial Immune System, Network Anomaly Detection

With accelerating informatization and digitalization, cybersecurity threats in cyberspace have intensified significantly. According to the Kaspersky Security Bulletin 2024, global malware attacks reached 302 million incidents—averaging approximately 820,000 detections daily¹. In this context, Network Anomaly Detection Systems (NADS), also termed Network Intrusion Detection Systems (NIDS), monitor malicious activities and policy violations within networks to provide robust protection against cyberattacks². As a critical cybersecurity infrastructure component, network anomaly detection constitutes an indispensable element of network behavior analysis and represents an active research domain in both academia and industry³. Contemporary methodologies addressing this challenge include Artificial Neural Networks⁴, Support Vector Machines (SVM)⁵, K-Nearest Neighbors (KNN)⁶, clustering techniques⁷, deep learning architectures⁸, Bayesian networks⁹, and negative selection algorithms¹⁰, among others.

This study focuses on the application of the Negative Selection Algorithm (NSA) in network anomaly detection. Inspired by the human immune system, which protects the body by distinguishing self from non-self to identify and eliminate foreign pathogens¹¹, network anomaly detection operates on a strikingly similar principle. Both systems maintain stability within dynamic environments by differentiating between normal (self) and anomalous (non-self) entities. Consequently, the NSA, derived from this biological paradigm, has been considered particularly well-suited for network anomaly detection¹². Within the human immune system, immature T cells undergo a process of negative selection (self-tolerance) in the thymus to achieve maturity. T cells that react strongly with self-molecules are eliminated, thereby preventing mature T cells from mistakenly attacking the body's own tissues. Mature T cells exiting the thymus are thus tolerant to self but capable of recognizing non-self threats. In 1994, Professor Stephanie Forrest at the University of New Mexico proposed the Negative Selection Algorithm¹³. This algorithm simulates the T-cell maturation mechanism to generate detectors capable of identifying network anomalies. Figure 1 illustrates the core concepts of the NSA¹⁴.

As illustrated in Figure 1, the NSA comprises two fundamental phases: training and detection. During the training phase, randomly generated candidate detectors undergo self-tolerance exposure, eliminating those detectors that recognize any self-antigens; only candidates demonstrating no self-reactivity mature into functional detectors and join the detector set for subsequent non-self identification. In the detection phase, test

School of Engineering, Honghe University, Mengzi, Yunnan Province 661199, China. ✉email: lizhiyong@uoh.edu.cn

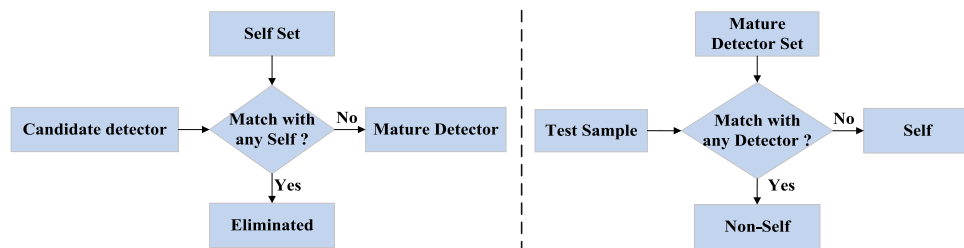


Fig. 1. Core Concepts of the Negative Selection Algorithm.

samples matching (i.e., being covered by) any mature detector are classified as non-self, while samples failing to match any detector in the set are classified as self. Crucially, the NSA training phase requires only self-samples yet enables the detection of non-self anomalies. This distinctive feature underpins NSA's widespread application in network anomaly detection¹⁰, network intrusion detection¹⁵, fault detection¹⁶, civil engineering structural health monitoring¹⁷, and related domains. Fundamentally, these applications constitute binary classification problems distinguishing self (normal) from non-self (abnormal)¹⁸.

Although the NSA has gained widespread adoption in network anomaly detection, its application faces two fundamental limitations in high-dimensional feature spaces: detector generation difficulty and the hole problem. Traditional NSA employs random candidate detector generation, which performs adequately in low-dimensional spaces with uniform sample distribution. However, in high-dimensional spaces like the NSL-KDD dataset, sample distribution becomes sparse and complex, with most samples concentrated in low-dimensional subspaces. The randomness of detector generation proves inefficient here, as candidate detectors rarely land in the relevant feature subspaces where actual samples reside. Consequently, most randomly generated detectors are eliminated during training, resulting in insufficient mature detectors and algorithm failure.

To address the detector generation challenge, we propose using abnormal samples from the training set as detector centers. Since these samples encapsulate authentic feature space distribution characteristics, they enable targeted detector generation within the actual sample subspaces. Notably, while traditional NSA utilizes only normal samples for training, our approach incorporates both normal and abnormal samples.

Regarding the hole problem—uncovered regions where samples are misclassified as normal—we implement a nearest-neighbor reclassification strategy. Samples within holes are reclassified based on the class attributes of their nearest neighbors, thereby reducing misclassification of abnormal samples in these uncovered regions.

To summarize, the principal contributions of this work are:

- (1) **Abnormal sample-guided detector generation:** Departing from traditional NSA's random detector generation, we propose using abnormal training samples as detector centers. These samples encapsulate authentic feature space distribution characteristics, effectively mitigating the detector generation challenge in high-dimensional spaces.
- (2) **Hole remediation via neighbor classification:** Replaces traditional coverage reduction with nearest-neighbor reclassification of uncovered samples, reducing abnormal misidentifications.
- (3) **Comparative Performance Validation:** Demonstrates statistically significant superiority over eight baseline algorithms on benchmark datasets NSL-KDD and UNSW-NB15.

The remainder of this paper is structured as follows. Section 2 provides a concise overview of NSA and relevant literature. Section 3 details the limitations of traditional NSA and presents our proposed solutions. Section 4 validates the proposed algorithm through experiments on NSL-KDD and UNSW-NB15 datasets with comparative analysis against eight baseline methods. Section 5 concludes the study and suggests future research directions.

Related work

The historical evolution of negative selection algorithms commenced in 1994 when Professor Stephanie Forrest pioneered the negative selection algorithm, generating detectors that simulate T-cell maturation mechanisms to identify network abnormalities¹³. Subsequent development occurred in 2000 with Hofmeyr and Forrest's Artificial Immune System (ARTIS) for network intrusion detection, utilizing binary representations of antibodies/antigens and r-continuous bit matching for similarity measurement¹⁹. A significant advancement emerged in 2003 through González et al.'s Real-Valued Negative Selection Algorithm (RNSA), which normalized antigens and antibodies to unit hypercube space $[0, 1]^n$ and employed Minkowski distance metrics, thereby extending NSA to real-valued applications²⁰. This progression culminated in 2004 with Ji and Dasgupta's V-Detector algorithm featuring variable-radius detectors, where dynamically adjusted radii enable large detectors to cover extensive non-self regions while compact detectors address coverage gaps (holes)²¹. Figure 2 visually contrasts non-self space coverage between fixed-radius and variable-radius detectors in 2D feature space: green circles denote self-regions and blue circles represent detectors, demonstrating how variable-radius detectors achieve superior non-self coverage with greater efficiency.

Following pivotal transitions from binary to real-valued space and fixed to variable-radius detectors, NSAs have gained substantial adoption in network anomaly detection due to their capability to identify unknown attacks without prior knowledge. Subsequent innovations include Gong et al. (2012) introducing self-region

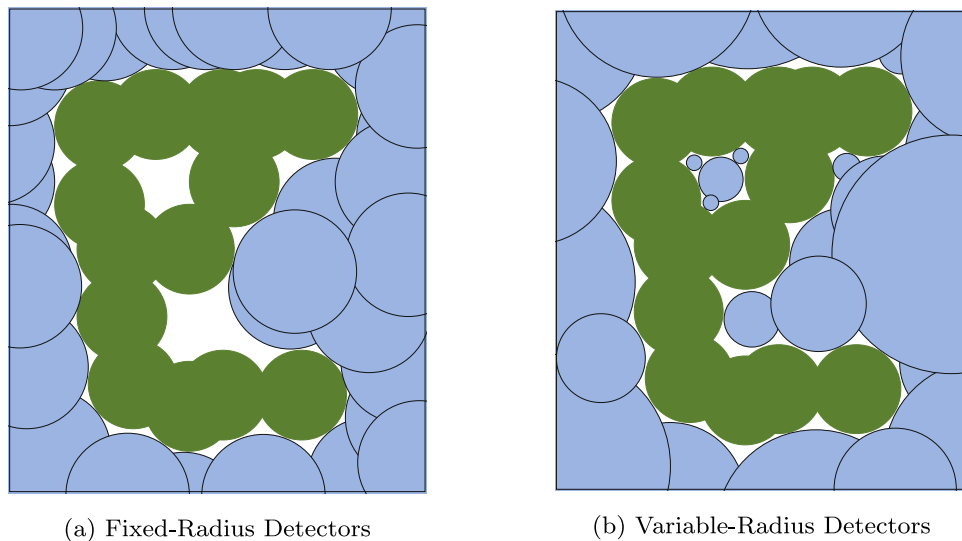


Fig. 2. Coverage Performance Comparison Between Fixed-Radius and Variable-Radius Detectors.

covering detectors to reduce training time while improving detection rates through Euclidean distance-based hole classification²²; Chen et al. (2013) developing hierarchical clustering-based CB-RNSA²³; Idris (2014) optimizing detectors via particle swarm optimization²⁴; Lin et al. (2015) proposing bidirectional inhibition-based BIORV-NSA for enhanced coverage efficiency²⁵; Saurabh and Verma (2016) creating EPAADPS with self-tuning evolutionary detectors²⁶; Tao et al. (2017) designing antigen density-driven ASD-RNSA with antibody suppression termination criteria²⁷; Ziwen et al. (2018) enhancing V-Detector with phased generation (Prophase large-radius/Anaphase small-radius detectors) for wireless sensor networks²⁸; Weikai et al. (2019) developing IDIA combining shift-mutation NSA with dynamic clonal selection for industrial cloud security²⁹; Chao et al. (2020) creating ADC-NSA using non-self cluster centers/outliers for detector generation³⁰; Liang et al. (2021) proposing multisource detectors via chaotic/DNA-GA methods³¹; Kim et al. (2022) implementing hybrid NSA/clone selection for industrial multi-class detection³²; and Hanyuan et al. (2023) establishing hierarchical differential evolution-based intrusion detection³³. XiaoWen et al. (2024) addressed the hole problem through self-antigen-based candidate detectors with complementary space representation³⁴, while Lu Peng et al. (2025) employed multi-layer immune tolerance to reduce false positives³⁵.

While numerous scholars have successfully applied NSA to network anomaly detection with significant results, two fundamental challenges persist: detector generation difficulties in high-dimensional feature spaces and the unresolved hole problem. Previous approaches to dimensionality challenges primarily employed reduction techniques-including principal component analysis^{26,28–30}, feature extraction³¹, and feature selection³⁶—though earlier studies^{23–25} lacked validation on standard datasets like KDD CUP99 and NSL-KDD. Our methodology addresses the detector generation problem by utilizing anomaly samples from training data as detector centers. Since these anomalies naturally cluster in low-dimensional subspaces, this approach enables efficient detector generation within their actual distribution regions. Crucially, unlike traditional NSA that exclusively uses normal samples during training, our algorithm effectively leverages abnormal samples. Regarding the hole problem, where prior solutions focused on coverage reduction^{24,27,28,30,31}, we implement nearest-neighbor reclassification based on adjacent sample attributes to mitigate abnormal sample misclassification in uncovered regions.

The proposed algorithm

In this section, we first analyze limitations in traditional negative selection algorithms, then present our proposed solution.

Limitations of the traditional negative selection algorithm

As established, the dual challenges of detector generation in high-dimensional feature spaces and the hole problem persist in current NSAs. We address these fundamental limitations separately in the following subsections.

High-dimensional detector generation

We first define the feature subspaces, then analyze the distribution patterns of samples in the NSL-KDD dataset, and finally demonstrate that randomly generated candidate detectors exhibit low probability of occupying the low-dimensional subspaces where samples predominantly reside. This empirically confirms that samples concentrate in low-dimensional feature subspaces while candidate detectors distribute sparsely across high-dimensional space, fundamentally impeding effective detector generation in high-dimensional feature spaces.

Subspace Definition Let feature space U have dimension n . A subspace $S \subseteq U$ is termed:

An n -dimensional subspace if $\dim(S) = n$ and $S \subsetneq U$ (proper subset)

A low-dimensional subspace if $\dim(S) < n$

Where U is a vector space over field $\mathbb{F}$, and subspaces inherit the ambient space's metric structure.

For a sample $S = (s_1, s_2, \dots, s_n)$ in the feature space U , if the sample's value is greater than zero in every dimension (i.e., $\forall s_i > 0, i = 1, 2, \dots, n$), then the sample lies in an n -dimensional subspace. Otherwise, if there is at least one dimension with a value of zero (i.e., $\exists s_i = 0, i = 1, 2, \dots, n$), then the sample lies in a low-dimensional subspace of U .

NSL-KDD, a well-known dataset for evaluating network anomaly detection algorithms, is partitioned into a training set (KDD Train+) containing 125,973 records and a test set (KDD Test+) containing 22,544 records; we performed statistical analysis on all sample values across dimensions in the KDD Train+ training set, with results presented in Table 1.

As shown in Table 1, within the 41-dimensional KDD Train+ training set, the “num_outbound_cmds” attribute has a value of zero for all samples; furthermore, attributes “is_host_login”, “urgent”, “land”, “num_shells”, and “su_attempted” exhibit zero values for most samples, indicating that all samples lie primarily in a low-dimensional subspace. However, traditional NSAs generate candidate detectors randomly and fail to target these specific low-dimensional feature subspaces where samples reside, leading to excessive pruning during training that prevents their maturation.

Corollary 1 As dimension $n \rightarrow \infty$, the probability $P_{j \dots k}$ that a randomly drawn candidate detector lies within a given target low-dimensional subspace $U_{s_j \dots s_k}$ approaches zero.

Proof Let $\mathbb{R}^n$ be an n -dimensional Euclidean space and $U_{s_j \dots s_k}$ be an s -dimensional subspace of $\mathbb{R}^n$, where $s < n$. Without loss of generality, assume $U_{s_j \dots s_k}$ is defined by the first s coordinate axes, i.e., $U_{s_j \dots s_k} = \mathbb{R}^s \times 0^{n-s}$. Suppose each component d_i ($i = 1, 2, \dots, n$) of a randomly generated candidate detector $D = (d_1, d_2, \dots, d_n)$ takes one of m discrete values, so the probability that $d_i = 0$ for any specific dimension i is $P_{d_i} = \frac{1}{m}$. Consequently, the probability $P_{j \dots k}$ that D lies within the target subspace $U_{s_j \dots s_k}$ is given by:

$$\begin{aligned} \lim_{n \rightarrow \infty} P_{j \dots k} &= \lim_{n \rightarrow \infty} \prod_{i=j}^k (P_{d_i \neq 0}) \prod_{i=1, i \neq j \dots k}^n (P_{d_i = 0}) \\ &= \lim_{n \rightarrow \infty} \prod_{i=j}^k \left(1 - \frac{1}{m}\right) \prod_{i=1, i \neq j \dots k}^n \left(\frac{1}{m}\right) \\ &= 0 \end{aligned}$$

Proved Corollary 1 demonstrates the difficulty for randomly generated candidate detectors to lie within the low-dimensional target subspace containing the samples. This constitutes the fundamental blind spot in random candidate detector generation. For a candidate detector to mature, it must undergo a self-tolerance process to eliminate those recognizing normal samples. Because randomly generated candidate detectors reside in the high-dimensional feature space while samples inhabit a low-dimensional feature subspace, the self-tolerance process removes a vast number of candidates. Consequently, the algorithm fails due to an insufficient number of mature detectors produced.

The hole problem in high-dimensional feature spaces

We first define the concept of holes, then analyze why they readily form at class domain boundaries and demonstrate their inevitability using the Haberman's Survival dataset. Finally, we explain that the hole problem becomes significantly more serious as the dimensionality of the feature space increases.

Holes are regions within the feature space that remain uncovered by any detector. The negative selection algorithm aims to generate detectors that cover as much non-self space (representing abnormal samples) as possible while avoiding coverage of self space (normal samples). However, due to the inherent irregularity of the boundary separating self and non-self regions, detectors struggle to achieve complete non-self coverage without inadvertently overlapping self areas, thereby creating holes. This problem is particularly pronounced where self and non-self regions intersect and overlap. We illustrate this phenomenon using the Haberman's Survival dataset.

Figure 3(a) shows the sample distribution of Haberman's Survival dataset. Blue asterisks (*) represent self samples, while red circles (o) represent non-self samples. As illustrated, samples are concentrated in the bottom region, where normal and abnormal samples overlap significantly. In contrast, samples in the top region are sparsely distributed.

Consequently, detectors generated in the top region can utilize larger radii to cover more non-self space, resulting in fewer holes. However, in the dense bottom region, only small-radius detectors can be generated. This leaves substantial non-self space uncovered, leading to more holes.

Figure 3(b) displays the detector distribution generated by the V-Detector algorithm on this dataset. It demonstrates the expected pattern: detectors in the top region have large radii and few holes, while those in the bottom region have small radii and many holes.

The Haberman's Survival dataset example illustrates that holes tend to form in boundary regions between normal and abnormal samples. This problem becomes particularly severe when these sample categories intersect or overlap. As feature space dimensionality increases, sample distribution becomes sparse, making it increasingly difficult to cover the entire non-self region with a finite number of detectors. Even with more detectors, coverage density may still decrease relative to the exponential growth of space volume.

When covering non-self space with hypersphere detectors in n -dimensional space, the volume V of any given hypersphere is given by the well-known formula:

Number	Attributes	The number of samples that take a value of zero on this attribute
1	<i>duration</i>	115955
2	<i>protocol_type</i>	0
3	<i>service</i>	0
4	<i>flag</i>	0
5	<i>src_bytes</i>	49392
6	<i>dst_bytes</i>	67967
7	<i>land</i>	125948
8	<i>wrong_fragment</i>	124883
9	<i>urgent</i>	125964
10	<i>hot</i>	123302
11	<i>num_failed_logins</i>	125851
12	<i>logged_in</i>	76121
13	<i>num_compromised</i>	124687
14	<i>root_shell</i>	125804
15	<i>su_attempted</i>	125893
16	<i>num_root</i>	125324
17	<i>num_file_creations</i>	125686
18	<i>num_shells</i>	125926
19	<i>num_access_files</i>	125602
20	<i>num_outbound_cmds</i>	125973
21	<i>is_host_login</i>	125972
22	<i>is_guest_login</i>	124786
23	<i>count</i>	13
24	<i>srv_count</i>	13
25	<i>error_rate</i>	86829
26	<i>srv_error</i>	88754
27	<i>error_rate</i>	109783
28	<i>srv_error_rate</i>	109767
29	<i>same_srv_rate</i>	2766
30	<i>diff_srv_rate</i>	76217
31	<i>srv_diff_host_rate</i>	97574
32	<i>dst_host_count</i>	3
33	<i>dst_host_srv_count</i>	3
34	<i>dst_host_same_srv_rate</i>	6927
35	<i>dst_host_diff_srv_rate</i>	46989
36	<i>dst_host_same_src_port_rate</i>	63023
37	<i>dst_host_srv_diff_host_rate</i>	86904
38	<i>dst_host_error_rate</i>	81386
39	<i>dst_host_srv_error_rate</i>	85360
40	<i>dst_host_error_rate</i>	103178
41	<i>dst_host_srv_error_rate</i>	106616

Table 1. Statistical Summary of KDD Train+ Samples.

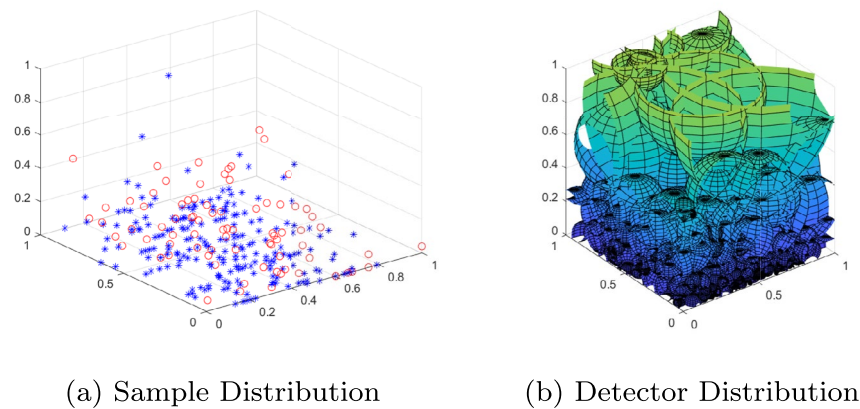


Fig. 3. Haberman's Survival Dataset.

$$V(r) = \frac{\pi^{n/2}}{\Gamma\left(\frac{n}{2} + 1\right)} r^n \quad (1)$$

Where r is the hypersphere radius and Γ denotes the gamma function. If we cover the non-self space with k such hyperspheres, the total coverage volume is $kV(r)$.

To completely cover an n -dimensional unit hypercube (of volume $1^n = 1$), the following condition must be satisfied:

$$kV(r) \geq 1 \quad (2)$$

Since hypersphere volume decays rapidly with increasing dimensionality n , the number of detectors k required for equivalent coverage grows exponentially. Furthermore, mutual detector overlap makes it prohibitively difficult to cover the entire non-self region with finite detectors in high-dimensional space, inevitably leaving numerous coverage holes.

Our solution

We contend that the randomness-induced inefficiency in candidate detector generation constitutes the primary challenge for detector formation in high-dimensional feature spaces. Randomly generated candidate detectors occupy high-dimensional subspaces, while actual samples reside on lower-dimensional manifolds within this space. This dimensional mismatch prevents most candidate detectors from maturing into functional detectors, causing algorithm failure due to insufficient mature detector counts. To address this, we propose using training-set abnormal samples as candidate detector centers. This approach not only eliminates the blindness of random generation but also yields mature detectors capable of effectively detecting surrounding anomalies, as these abnormal samples encode authentic distributional characteristics of the feature subspace. Crucially, whereas traditional NSAs utilize only normal samples during training, our method incorporates both normal and abnormal samples-aligning with established machine learning methodologies including random forests, K-nearest neighbors, and support vector machines. Thus, our algorithm effectively leverages the previously untapped potential of abnormal training samples ignored by conventional NSAs.

Regarding the hole problem, we contend that the misclassification of abnormal samples within uncovered regions as normal constitutes the primary cause of performance degradation. Since holes remain uncovered by detectors, all contained samples—including abnormal instances—are misclassified as normal. To mitigate this, we reclassify samples in uncovered regions based on the class labels of their nearest neighbors. This approach addresses the fundamental nature of hole formation: holes predominantly emerge in boundary regions between normal and abnormal classes, with severity exacerbating where these classes intersect or overlap. KNN leverages local similarity principles to generate naturally adaptive decision boundaries in class-overlap regions, circumventing the geometric constraints that cause structural deviations in linear models on complex frontiers; as established in machine learning literature, “the local decision characteristics of KNN enable exceptional performance in class overlap regions, particularly under highly nonlinear decision boundaries⁴⁵,” with the method intrinsically deriving classification decisions from neighboring sample distributions rather than predetermined geometric separators—yielding superior robustness in high-ambiguity regions where traditional boundary-dependent classifiers exhibit significant performance degradation.

The algorithm flowchart is depicted in Figure 4. Our approach comprises two sequential stages: NSA classification and KNN classification. During the NSA stage, we utilize abnormal training samples as candidate detector centers for generating mature detectors; candidates not covered by existing mature detectors or normal samples ultimately mature into detectors. These detectors collectively form the detector set for test evaluation: samples uncovered by any detector are labeled normal, while covered samples are classified as abnormal. Since holes remain uncovered by detectors, all contained samples—including anomalies—are misclassified as normal.

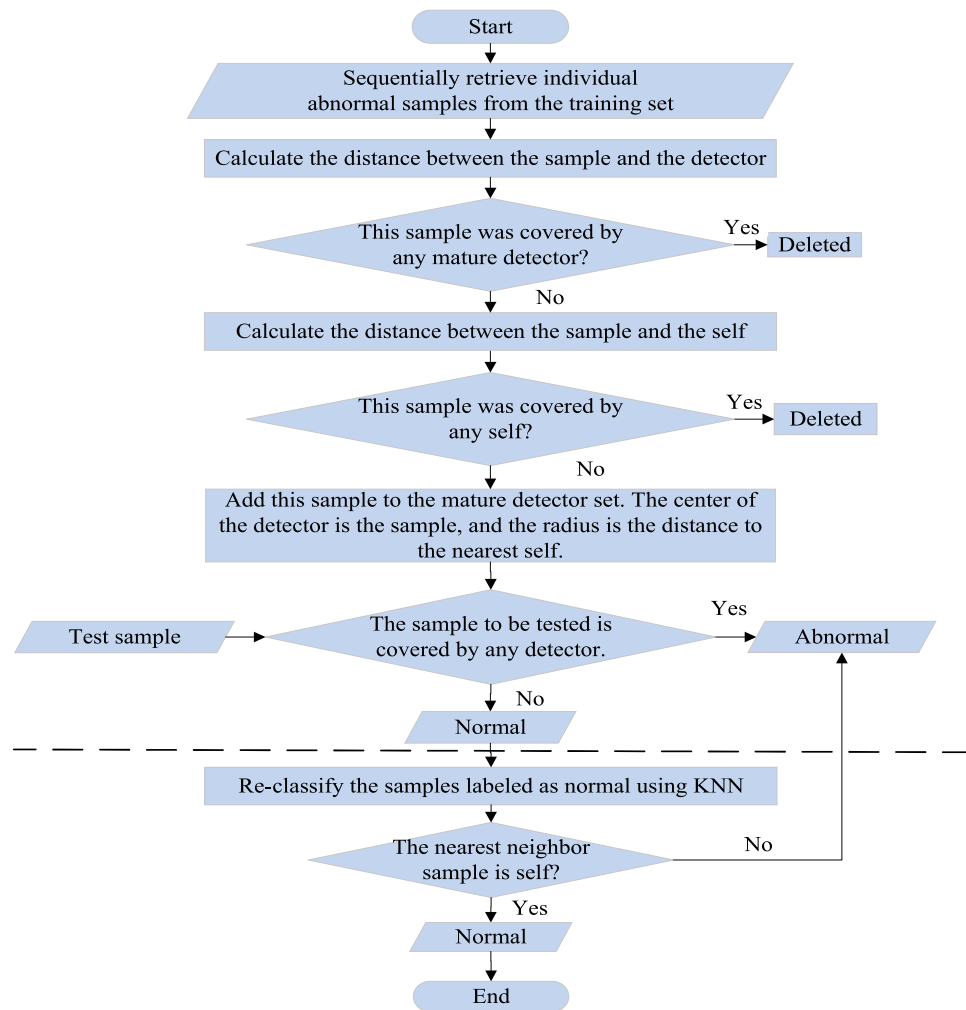


Fig. 4. Proposed Algorithm Flowchart.

To mitigate this, we introduce KNN reclassification for NSA-labeled normal samples. Detailed algorithmic implementation follows.

Data preprocessing

Core steps in data preprocessing are numerical encoding and normalization.

(1) Numerical Encoding

The dataset contains both quantitative (numeric) and qualitative (symbolic) features. Since our algorithm relies on Euclidean distance for sample similarity measurement – a metric applicable only to numerical values – we unify feature formats by mapping symbolic features to numerical representations. This process, known as numerical encoding (or feature conversion), transforms categorical attributes into numerical equivalents. For instance, the non-numeric features “*protocol_type*”, “*service*”, and “*flag*” in the NSL-KDD dataset undergo this conversion, as illustrated in Figure 5².

As shown in Figure 5, we mapped symbolic features to numerical representations by assigning specific values: for “*protocol_type*”, TCP=0, ICMP=0.5, UDP=1; similarly, the symbolic attack type (class) was converted with Normal=1, DoS=2, Probe=3, R2L=4, and U2R=5.

(2) Normalization

Normalization scales feature values into a specific interval (e.g., [0,1]), primarily eliminating bias from raw data while preserving statistical characteristics². Common methods include linear transformation and z-score standardization, as defined in Eq. (3) and Eq. (4) respectively.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (3)$$

where x represents the original feature values, $\min(x)$ and $\max(x)$ denote the minimum and maximum values of x respectively, and x' is the normalized output.

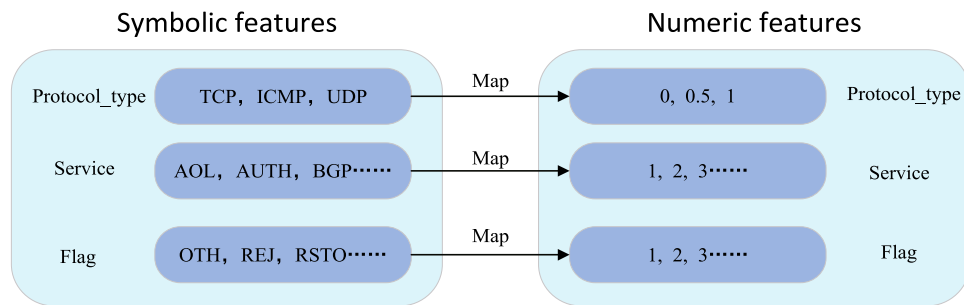


Fig. 5. Numerical Encoding of Categorical Features in the NSL-KDD Dataset.

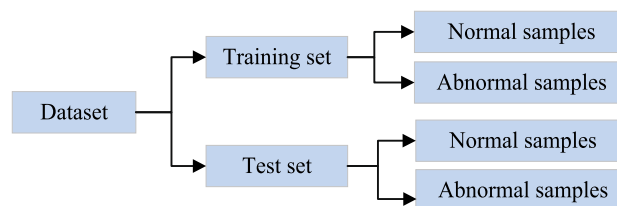


Fig. 6. Training and Test Sets.

$$z = \frac{x - \mu}{\sigma} \quad (4)$$

where x represents the feature values, μ denotes their mean, and σ is their standard deviation.

(3) Distance Metric

Euclidean distance, commonly employed by NSAs and KNN for similarity quantification, measures the separation between vectors $x = \{x_1, x_2, \dots, x_n\}$ and $y = \{y_1, y_2, \dots, y_n\}$ in n -dimensional space as

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (5)$$

Network anomaly detection via negative selection algorithm classification

Our algorithm replaces random candidate detector generation by directly using training set anomalies as detector centers to generate mature detectors.

(1) Training and Test Sets

As illustrated in Figure 6, datasets are conventionally partitioned into training and test sets. Traditional NSAs exclusively utilize normal training samples for detector generation, discarding valuable abnormal samples. Our algorithm overcomes this limitation by leveraging both normal and abnormal training samples.

(2) Detector Generation

Our algorithm employs abnormal training samples as candidate detector centers for generating mature detectors through a dual-tolerance process: First, candidate detectors undergo tolerance with existing mature detectors, where any candidate covered by mature detectors is eliminated to prevent coverage redundancy; second, remaining candidates undergo tolerance with normal samples, eliminating those matching normal patterns to avoid false positives. Only candidates surviving both stages become mature detectors, which are added to the detector set for anomaly identification, as formalized in Algorithm 1.

Abnormal_set: The set of abnormal samples in the training set

Normal_set: The set of normal samples in the training set

r_s : self radius

$r \leftarrow infinite$

- 1 $D \leftarrow \emptyset$ //Initialize the detector set to an empty set
- 2 Repeat for every $abnormal_j$ in $Abnormal_set = \{abnormal_j, j = 1, 2, \dots\}$
//Take abnormal samples one by one
- 3 Repeat for every d_i in $D = \{d_i, i = 1, 2, \dots\}$ //Comparing candidate detectors to existing mature detectors
- 4 $d_d \leftarrow$ Euclidean distance between d_i and $abnormal_j$
- 5 if $d_d \leq r(d_i)$, where $r(d_i)$ is the radius of d_i , then
- 6 go to 2
- 7 Repeat for every $normal_i$ in $Normal_set = \{normal_i, i = 1, 2, \dots\}$ //Comparing candidate detectors to self
- 8 $d \leftarrow$ Euclidean distance between $normal_i$ and $abnormal_j$
- 9 if $d - r_s \leq r$, then $r \leftarrow d - r_s$
- 10 if $r > r_s$, then $D \leftarrow D \cup \{ \langle abnormal_j, r \rangle \}$, where $\langle abnormal_j, r \rangle$ is a detector
- 11 return D

Algorithm 1. uses anomaly samples as candidate detector centers to train and generate mature detectors

(3) Anomaly Classification via Negative Selection

The mature detectors generated by Algorithm 1 form the detector set used to classify test samples. Samples covered by any detector are classified as anomalies; otherwise, they are designated normal. Defining *Label_Abnormal* as the set of test samples classified anomalous and *Label_Normal* as those classified normal, we establish:

$$Label_Normal \cup Label_Abnormal = Testset, Label_Normal \cap Label_Abnormal = \emptyset \quad (6)$$

The test set classifications yield normal and abnormal predictions, with the four possible outcomes detailed in Table 2: True Positives (TP) indicate correct classification of normal samples; False Positives (FP) denote normal samples misclassified as abnormal; False Negatives (FN) represent abnormal samples misclassified as normal; True Negatives (TN) reflect correct identification of abnormal samples. Among these, TP and TN signify correct classifications, while FP and FN indicate errors. For NSAs, FP primarily arises from incomplete normal samples in the training set, causing mature detectors to cover normal patterns. However, in benchmark datasets like NSL-KDD - where predefined training/test splits ensure consistent comparison - we consider the training set's normal samples complete. Thus, minimal FP rates should be algorithmically acceptable. Conversely, FN stems from coverage holes: regions uncovered by detectors cause all contained samples (including anomalies) to be classified as normal. To mitigate FN errors, we introduce KNN to reclassify NSA-classified normal samples, reducing such misclassifications.

		Actual	
		Normal	Abnormal
Predicted	Normal	TN	FP
	Abnormal	FN	TP

Table 2. Binary Classification Confusion Matrix.

False negative correction via KNN reclassification

“Holes” denote uncovered regions in the feature space where no mature detectors provide coverage. These emerge primarily at self/non-self boundaries due to: 1) self-radius variability, 2) irregular self-region geometries, 3) the detector exclusion constraint around self-samples, and 4) finite detector counts. Since holes lack detector coverage, all contained samples are misclassified as normal (self), causing anomalous (non-self) samples to evade detection. The KNN algorithm addresses this limitation by classifying samples based on nearest-neighbor proximity rather than explicit class boundaries. This approach proves particularly effective for boundary regions with class-domain overlap - precisely where holes form - outperforming boundary-dependent classifiers. The KNN training and test data configuration is illustrated in Figure 7.

As depicted in Figure 7, the NSA algorithm partitions test set samples into *Label_Normal* and *Label_Abnormal*. Given our prior justification of acceptable false positive rates under complete training set normal samples, *Label_Abnormal* requires no reclassification. Conversely, coverage holes in *Label_Normal* contain misclassified anomalies. Thus, we deploy KNN exclusively on *Label_Normal* to identify these false negatives. Formally, KNN training utilizes $TrainingSet \cup Label_Abnormal$ with *Label_Normal* as its test subset.

Analysis of algorithm complexity

Our algorithm generates detectors from anomalous training samples, yielding a training time complexity of:

$$O\left(\frac{N_d * N_s}{(1 - P_m)^{N_s}}\right) \quad (7)$$

where N_d denotes the detector count, N_s represents the self element count, and P_m indicates the antigen-detector match probability³⁷.

During detection, each test sample must be matched against all detectors, resulting in a time complexity of:

$$O(N_d * N) \quad (8)$$

where N denotes the number of test samples.

KNN performs reclassification for false-negative correction, exhibiting time complexity:

$$O(N_s * D) \quad (9)$$

where N_s denotes test sample cardinality and D represents feature dimensionality.

Experimental evaluation

To validate the effectiveness of our proposed method, we conducted experiments on the NSL-KDD³⁸ and UNSW-NB15^{39,40} datasets.

Evaluation metrics

The six evaluation metrics used in the experiment are: detection rate (DR), false alarm rate (FAR), accuracy, F1-score, number of mature detectors (ND), detector training time, and detection time.

Detection Rate (DR), also known as True Positive Rate (TPR) or Recall, evaluates intrusion detection system (IDS) performance by measuring the proportion of correctly identified attacks among all actual attacks. Formally, it is defined as the ratio of true positives (correctly detected attacks) to the total number of actual attacks.

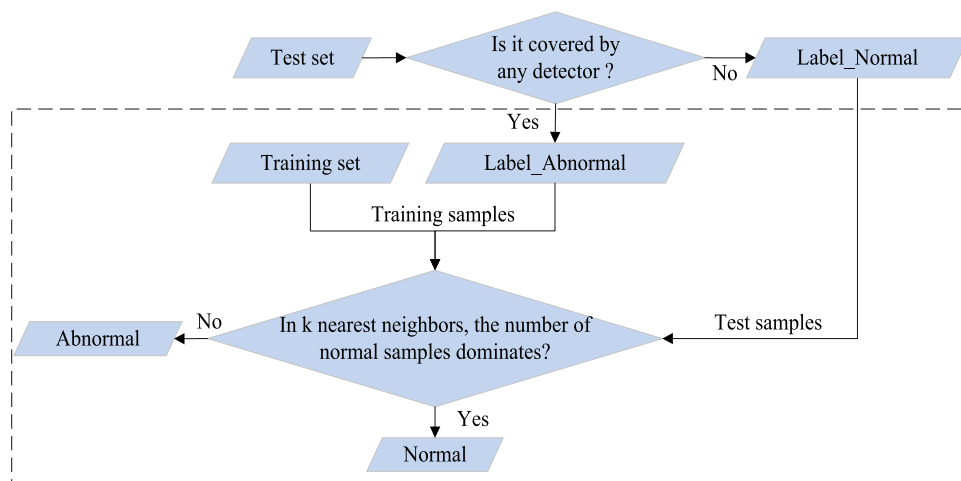


Fig. 7. KNN Training/Test Set Composition.

parameter	default value
r_s (self radius)	0.01
P_{exp} (expected coverage rate)	0.99
MSC (Maximum Self Coverage)	0.99
Max_D (Maximum number of detectors)	1000

Table 3. The default value of the parameter of the NSAs.

value of k	Detection rate	False alarm rate	Accuracy	Detection time(s)
$k = 1$	64.47%	2.27%	78.80%	696.8
$k = 3$	64.33%	2.35%	78.68%	672.8
$k = 5$	61.90%	2.27%	77.33%	609.7
$k = 7$	60.38%	2.34%	76.44%	609.1
$k = 9$	60.04%	2.34%	76.25%	608.6
$k = 11$	59.64%	2.38%	76.00%	609.5
$k = 13$	59.64%	2.41%	75.98%	609.4
$k = 15$	59.95%	2.44%	76.15%	608.4
$k = 17$	60.19%	2.42%	76.30%	671.5
$k = 19$	60.66%	2.46%	76.55%	783.5

Table 4. Classification Results of NSL-KDD Dataset Using KNN with Varying k Values.

$$DR = \frac{TP}{TP + FN} \quad (10)$$

The False Positive Rate (FPR), or probability of false alarm, quantifies an intrusion detection system's (IDS) tendency to misclassify normal activities as attacks. Formally, FPR is defined as the ratio of false positives (normal activities incorrectly flagged as attacks) to the total number of normal activities (i.e., false positives plus true negatives).

$$FAR = \frac{FP}{FP + TN} \quad (11)$$

The accuracy of an Intrusion Detection System (IDS) measures its overall correctness in distinguishing between attacks and normal activities. It is defined as the ratio of correct predictions—the sum of true positives (correctly flagged attacks) and true negatives (correctly identified normal activities)—to the total number of predictions made, which includes true positives, true negatives, false positives (normal activities incorrectly flagged as attacks), and false negatives (attacks missed).

$$\text{Accuracy} = \frac{TN + TP}{TP + FN + FP + TN} \quad (12)$$

The F1-score is a performance metric used to evaluate classification models, particularly in binary classification tasks. It represents the harmonic mean of precision and recall, providing a single balanced measure of both metrics. The formula for calculating the F1-score is:

$$F1 = \frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}} \quad (13)$$

where precision is calculated using the following formula:

$$\text{precision} = \frac{TP}{TP + FP} \quad (14)$$

Table 2 in Section 3.2.2 defines the terms TP, TN, FN, and FP as used in Equations (10)–(14).

Parameter configuration

The experiment employed eight comparative algorithms: V-Detector²¹, BIORV-NSA²⁵, Improved V-Detector²⁸, ADC-NSA³⁰, KNN⁴¹, SVM⁴², Decision Trees⁴³, and Random Forest⁴⁴. The first four represent NSAs, while the latter four are traditional machine learning methods.

Algorithm	DR(%)		FAR(%)		Accuracy(%)		F1		Training Time(s)		Detection Time(s)		Number of detectors	
	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD
V-Detector	0.14	0.17	0.02	0.06	43.15	0.10	0.003	0.003	0.78	0.37	0.30	0.17	1.6	0.7
BIORV-NSA	9.91	1.37	1.00	1.04	48.86	0.68	0.18	0.018	947.95	11.22	177.41	5.10	1000	0
Improved V-Detector	1.51	3.84	0.03	0.07	43.92	2.19	0.011	0.013	47.23	0.84	0.27	0.12	1.6	0.7
ADC-NSA	0.26	0.53	0	0	43.22	0.30	0.016	0.020	0.68	0.22	0.25	0.08	1.6	0.5
KNN	64.47	0	2.27	0	78.80	0	0.776	0			696.83	12.76		
SVM	48.80	0	1.2	0	70.33	0	0.652	0	123.50	3.21	0.87	0.11		
Decision Trees	62.67	0	3.12	0	77.40	0	0.76	0	1.326	0.075	0.004	0.0002		
Random Forest	62.13	1.88	2.89	0.13	77.20	1.09	0.758	0.0163	2.20	0.058	0.08	0.003		
Our algorithm	73.45	0.35	2.87	0.12	83.65	0.23	0.837	0.002	333.38	21.81	1006.76	68.84	586.2	7.7

Table 5. Performance Evaluation of Nine Algorithms Using NSL-KDD Dataset.

Algorithm	DR(%)		FAR(%)		Accuracy(%)		F1		Training Time(s)		Detection Time(s)		Number of detectors	
	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD
V-Detector	0.27	0.76	0.03	0.07	45.08	0.42	0.01	0.02	0.54	0.35	0.81	0.50	1.7	1
BIORV-NSA	8.61	0.91	0.49	0.19	49.46	0.53	0.158	0.016	1000	0	1267	97.1	631.6	47.3
Improved V-Detector	0.03	0.06	0.01	0.01	44.95	0.03	0.001	0.001	1.7	0.7	32.9	4.6	0.9	0.4
ADC-NSA	0.13	0.22	0.02	0.04	45.00	0.12	0	0	0.53	0.22	0.87	0.35	1.5	0.5
KNN	95.34	0	27.49	0	85.08	0	0.876	0			2868.2	117.8		
SVM	99.20	0	47.72	0	78.11	0	0.83	0	712.5	36.3	32.4	5.2		
Decision Trees	94.26	0	29.79	0	83.45	0	0.863	0	7.9	0.7	0.03	0.04		
Random Forest	98.72	0.08	34.00	0.91	84.01	0.39	0.872	0.003	4.4	1.0	0.3	0.1		
Our algorithm	98.79	0.02	31.51	0.07	84.90	0.04	0.879	0	8247.3	355.9	11511.2	631.2	16084.3	51.8

Table 6. Performance Evaluation of Nine Algorithms Using UNSW-NB15 Dataset.

The four parameters typically employed in NSAs are: self-radius (r_s), maximum number of detectors (Max_D), expected coverage (P_{exp}), and maximum self-coverage (MSC). Default values for these parameters are provided in Table 3.

Additionally, some negative-selection algorithms employ unique parameters. The BIORV-NSA²⁵ algorithm features two distinctive parameters: the self-set edge inhibition parameter ($\kappa = 0.8$) and antibody self-inhibition parameter ($\alpha = 1.2$). For the Improved V-Detector²⁸ algorithm, the detector redundancy thresholds $\delta_1 = 0.95$ and $\delta_2 = 1.2$ serve as unique parameters. The ADC-NSA³⁰ algorithm utilizes a single unique parameter: the cutoff distance ($dc = 0.25$).

For the four traditional machine learning algorithms, the KNN⁴¹ algorithm utilizes the number of nearest neighbors k as its key parameter; experimental results on the NSL-KDD dataset demonstrate optimal classification performance at $k = 1$, with comparative results for different k values provided in Table 4. The remaining algorithms - support vector machine (SVM)⁴², Decision Trees⁴³, and Random Forest⁴⁴ - employ default parameter configurations from MATLAB R2020b.

Our algorithm comprises four parameters: self-radius(r_s), maximum self-coverage(MSC), maximum number of detectors(Max_D), and the number of nearest neighbors(k). The first three parameters originate from the negative selection algorithm, as detailed in Table 3, while the last parameter corresponds to the KNN algorithm with $k = 1$.

Results and discussion

The study comprises two experiments: an anomaly detection experiment to evaluate the algorithm's ability to detect attacks, and an attack type detection experiment to assess its capability in identifying specific attack types.

Anomaly detection performance experiment

We conducted anomaly detection experiments on both the NSL-KDD and UNSW-NB15 datasets. Each experiment was repeated 10 times, with results summarized as means and standard deviations(SD) in Tables 5 and 6, respectively.

As shown in Table 5, our algorithm achieves superior performance with a detection rate of 73.45%, accuracy of 83.65%, and F1-score of 0.837, while maintaining a false positive rate (FPR) of 2.87% -lower than Decision Trees⁴³ and Random Forest⁴⁴. On 41-dimensional NSL-KDD data, detection rates for negative selection algorithms are substantially lower: V-Detector²¹ (0.14%), BIORV-NSA²⁵ (9.91%), Improved V-Detector²⁸ (1.51%), and ADC-NSA³⁰ (0.26%). The sub-2% detection rates of V-Detector, Improved V-Detector, and ADC-

Experiment	Detection rate	False alarm rate	Accuracy	Training time(s)	Detection time(s)	Number of detectors
1	84.38%	1.76%	92.22%	114.56	596.22	222.0
2	83.84%	1.86%	91.93%	110.04	578.30	217.0
3	85.02%	1.76%	92.50%	110.69	592.63	221.0
4	84.29%	1.80%	92.15%	108.51	575.42	216.0
5	84.22%	1.62%	92.23%	113.06	618.95	226.0
6	85.05%	1.78%	92.50%	111.55	631.37	230.0
7	84.81%	1.84%	92.36%	114.01	671.93	224.0
8	84.96%	1.77%	92.46%	111.61	614.17	220.0
9	84.73%	1.81%	92.34%	107.29	583.41	225.0
10	84.02%	1.77%	92.06%	107.82	653.47	225.0
Average	84.53%	1.78%	92.27%	110.91	611.59	222.6
Standard deviation	0.42%	0.06%	0.18%	2.40	31.07	4.1

Table 7. DOS Attack Detection Performance of Our Algorithm.

Experiment	Detection rate	False alarm rate	Accuracy	Training time(s)	Detection time(s)	Number of detectors
1	88.81%	2.23%	95.98%	107.83	404.91	215.0
2	89.96%	2.20%	96.23%	108.25	397.18	214.0
3	89.43%	2.59%	95.81%	111.80	416.54	220.0
4	90.13%	2.23%	96.24%	107.68	373.12	373.1
5	90.09%	2.67%	95.89%	108.93	414.25	219.0
6	87.48%	2.23%	95.71%	108.84	395.46	221.0
7	91.57%	2.27%	96.51%	104.86	395.35	212.0
8	88.97%	2.22%	96.02%	110.17	412.04	224.0
9	88.39%	2.22%	95.90%	111.16	411.22	226.0
10	86.58%	2.57%	95.26%	103.80	415.92	212.0
Average	89.14%	2.35%	95.96%	108.33	403.60	233.6
Standard deviation	1.36%	0.18%	0.32%	2.40	12.92	46.7

Table 8. Probe Attack Detection Performance of Our Algorithm.

NSA stem from severe detector insufficiency (average: 1.6 detectors), failing to cover non-self space. Although BIORV-NSA generates 1000 detectors (preset termination), its 9.91% detection rate reflects blindness in random candidate detector generation: detectors rarely populate the low-dimensional feature subspaces containing samples, preventing effective non-self identification. Among machine learning algorithms, KNN⁴¹, Decision Trees, and Random Forest all exceed 60% detection with 80% accuracy, while SVM⁴² achieves only 48.80%.

Table 6 presents UNSW-NB15 dataset results. Our algorithm achieves a detection rate of 98.79%, marginally lower than SVM (99.20%) but superior to the other seven algorithms. Despite SVM's detection rate advantage, its false alarm rate (47.72%) substantially exceeds our algorithm's 31.51%. In accuracy, our method reaches 84.90% – slightly below KNN – while outperforming KNN in detection rate. For F1-score, our algorithm demonstrates competitive advantage over all eight baselines. Regarding computational efficiency, our method shows no advantage in training/testing time.

Attack type detection experiment

The NSL-KDD dataset categorizes attacks into four types: DOS, Probe, U2R, and R2L. We further evaluated our algorithm's detection effectiveness against these attack categories.

(1) DOS Attack Detection Performance

We constructed the training set using normal and DOS attack samples from the NSL-KDD training partition, and the test set with normal and DOS samples from the test partition. Results are presented in Table 7.

For DOS attacks, our algorithm achieves an average detection rate of 84.53%, false positive rate of 1.78%, and accuracy of 92.27%, demonstrating effective detection capability. This performance stems from the clustered distribution of DOS attack samples. By utilizing training-set DOS samples as candidate detector centers, the algorithm generates detectors that efficiently identify DOS samples in test data.

(2) Probe Attack Detection Performance

We constructed the training set using normal and Probe attack samples from NSL-KDD's training partition, and the test set with corresponding samples from the test partition. Results appear in Table 8.

For Probe attacks, our algorithm achieves an average detection rate of 89.14%, accuracy of 95.96%, and false positive rate of 2.35%, demonstrating effective detection capability. This performance stems from the clustered

Experiment	Detection rate	False alarm rate	Accuracy	Training time(s)	Detection time(s)	Number of detectors
1	12.35%	0.13%	80.53%	91.81	455.76	133
2	12.38%	0.14%	80.53%	85.08	466.34	135
3	12.35%	0.13%	80.53%	88.25	469.25	139
4	12.31%	0.14%	80.51%	73.95	424.73	128
5	12.49%	0.13%	80.56%	84.16	461.72	132
6	12.42%	0.13%	80.55%	76.71	447.91	134
7	12.35%	0.13%	80.53%	90.87	464.57	133
8	10.53%	0.13%	80.13%	77.17	410.92	135
9	12.31%	0.14%	80.51%	73.78	435.41	128
10	12.42%	0.13%	80.55%	78.93	445.38	132
Average	12.19%	0.14%	80.49%	82.07	448.2	132.9
Standard deviation	0.56%	0.01%	0.12%	6.49	18.42	3.1

Table 9. R2L Attack Detection Performance of Our Algorithm.

Experiment	Detection rate	False alarm rate	Accuracy	Training time(s)	Detection time(s)	Number of detectors
1	8.00%	0.03%	98.11%	14.58	296.51	28
2	8.00%	0.03%	98.11%	15.85	305.25	30
3	8.00%	0.03%	98.11%	15.54	287.95	30
4	6.50%	0.03%	98.08%	14.22	308.96	27
5	6.50%	0.03%	98.08%	14.56	307.67	28
6	8.00%	0.03%	98.11%	14.08	284.26	27
7	6.50%	0.03%	98.08%	14.01	307.74	27
8	6.50%	0.03%	98.08%	13.65	310.08	26
9	8.00%	0.03%	98.11%	14.47	288.4	28
10	6.50%	0.03%	98.08%	14.57	329.85	27
Average	7.25%	0.03%	98.10%	14.55	302.67	27.8
Standard deviation	0.75%	0.00%	0.02%	0.64	12.99	1.2

Table 10. U2R Attack Detection Performance of Our Algorithm.

distribution of Probe attack samples in feature space. Leveraging training-set Probe samples as candidate detector centers, the generated detectors efficiently identify Probe samples in test data.

(3) R2L Attack Detection Performance

We constructed the training set exclusively using normal and R2L attack samples from NSL-KDD's training partition, with the test set derived similarly from the test partition. Results are presented in Table 9.

For R2L attacks, our algorithm yields an average detection rate of 12.19%, false positive rate of 0.14%, and accuracy of 80.49%. The high accuracy yet low detection rate primarily stems from severe class imbalance - normal samples substantially outnumber R2L samples. Under such imbalance, accuracy becomes an unreliable performance metric; detection rate better reflects correct classification of minority-class (R2L) samples. Crucially, the poor detection performance originates from the dispersed distribution of R2L samples in feature space, preventing trained detectors from effectively identifying test-set R2L samples.

(4) U2R Attack Detection Performance

We constructed the training set exclusively using normal and U2R attack samples from NSL-KDD's training partition, with the test set similarly composed of corresponding samples from the test partition. Results are presented in Table 10.

For U2R attacks, our algorithm yields an average detection rate of 7.25%, false positive rate of 0.03%, and accuracy of 98.10%. The high accuracy yet low detection rate is again attributable to severe class imbalance, with U2R samples significantly outnumbered by normal samples. Crucially, the poor detection performance originates from the dispersed distribution of U2R samples in feature space.

In summary, our algorithm demonstrates strong detection performance for DOS and Probe attacks, but performs poorly against R2L and U2R attacks. This differential performance directly correlates with sample distribution characteristics: attacks with clustered distributions achieve high detection rates, while those with dispersed distributions yield suboptimal results.

Conclusion

Traditional NSAs exclusively utilize normal samples for training, omitting anomaly samples. In this study, we leverage these previously unused anomaly samples as candidate detector centers to train detectors, mitigating high-dimensional detector generation challenges. Unlike traditional NSAs that employ blind random generation

– which struggles to produce effective detectors in low-dimensional feature subspaces where samples reside

– our approach uses training-set anomaly samples as detector centers. These provide authentic feature-space distribution information, enabling targeted detector generation in sample-dense subspaces. Critically, generated detectors effectively identify surrounding anomalies. In traditional NSAs, uncovered samples within sparse regions (“holes”) are classified as normal, leading to anomaly misclassification. To overcome this limitation, we assign classification based on the nearest neighbor’s category for hole-resident samples, reducing false normalization of anomalies. Comparative evaluation against eight baselines on benchmark intrusion detection datasets NSL-KDD and UNSW-NB15 validates our algorithm’s superiority.

Data availability

The data that support the findings of this study are available from the corresponding author, Zhiyong Li, upon reasonable request.

Received: 5 February 2025; Accepted: 15 September 2025

Published online: 17 October 2025

References

1. Kaspersky Security Bulletin 2024. Statistics, 4 DEC 2024. <https://securelist.com/ksb-2024-statistics/114795/>
2. Moustafa, N., Hu, J. & Slay, J. A holistic review of network anomaly detection systems: A comprehensive survey. *Journal of Network and Computer Applications* **128**, 33–55 (2019).
3. Fernandes, G. et al. A comprehensive survey on network anomaly detection. *Telecommunication Systems* **70**, 447–489 (2019).
4. Zhou, X. et al. Reconstructed graph neural network with knowledge distillation for lightweight anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems* **35**(9), 11817–11828 (2024).
5. Ashraf, M. W. A. et al. A hybrid approach using support vector machine rule-based system: detecting cyber threats in internet of things. *Scientific Reports* **14**(1), 27058 (2024).
6. Liao, Y. & Vemuri, V. R. Use of k-nearest neighbor classifier for intrusion detection. *Computers & security* **21**(5), 439–448 (2002).
7. Peng, K., Leung, V. C. M. & Huang, Q. Clustering approach based on mini batch kmeans for intrusion detection system over big data. *IEEE Access* **6**, 11897–11906 (2018).
8. Wang, S., Jiang, R., Wang, Z., et al. Deep learning-based anomaly detection and log analysis for computer networks. arXiv preprint [arXiv:2407.05639](https://arxiv.org/abs/2407.05639), (2024).
9. Faramondi, L. et al. A hybrid behavior-and Bayesian network-based framework for cyber-physical anomaly detection. *Computers and Electrical Engineering* **112**, 108988 (2023).
10. Gu, M. et al. A negative selection algorithm with hypercube interface detectors for anomaly detection. *Applied Soft Computing* **154**, 111339 (2024).
11. Li, Tao. *Computer Immunology[M]* (Electronic Industry Press, 2004).
12. Wang, W. et al. Intrusion detection and security calculation in industrial cloud storage based on an improved dynamic immune algorithm. *Information Sciences* **501**, 543–557 (2019).
13. Forrest, S., Perelson, A. S., Allen, L., et al. Self-nonsens discrimination in a computer[C]. In: Proceedings of the 1994 IEEE Symposium on Security and Privacy, IEEE Computer Society, (1994).
14. Dasgupta, D. Advances in Artificial Immune Systems. *IEEE Computational Intelligence Magazine* **1**(4), 40–49 (2006).
15. Chen, J., He, J., Li, W., et al. A hierarchical unmanned aerial vehicle network intrusion detection and response approach based on immune vaccine distribution. *IEEE Internet of Things Journal*, 2024.
16. Ren, Y., Wang, X. & Zhang, C. A Novel Fault Diagnosis Method based on Improved Negative Selection Algorithm. *IEEE Transactions on Instrumentation and Measurement* **70**, 1–8 (2020).
17. Barontini, A. et al. Deterministically Generated Negative Selection Algorithm for Damage Detection in Civil Engineering Systems. *Engineering Structures* **197**, 109444 (2019).
18. González, F. A. & Dasgupta, D. Anomaly detection using real-valued negative selection. *Genetic Programming and Evolvable Machines* **4**(4), 383–403 (2003).
19. Hofmeyr, S. A. & Forrest, S. Architecture for an artificial immune system. *Evolutionary computation* **8**(4), 443–473 (2000).
20. Gonzalez, F., Dasgupta, D. & Niño, L. F. A randomized real-valued negative selection algorithm[C]//International Conference on Artificial Immune Systems. Springer, Berlin, Heidelberg, 261–272. (2003).
21. Ji, Z. & Dasgupta, D. *Real-valued negative selection algorithm with variable-sized detectors[C]//Genetic and Evolutionary Computation Conference* 287–298 (Springer, 2004).
22. Gong, M. et al. An efficient negative selection algorithm with further training for anomaly detection. *Knowledge-Based Systems* **30**, 185–191 (2012).
23. Chen, W. et al. A negative selection algorithm based on hierarchical clustering of self set. *Science China Information Sciences* **56**, 1–13 (2013).
24. Idris, I. & Selamat, A. Improved email spam detection model with negative selection algorithm and particle swarm optimization. *Applied Soft Computing* **22**, 11–27 (2014).
25. Cui, L., Pi, D. & Chen, C. BIORV-NSA: Bidirectional inhibition optimization r-variable negative selection algorithm and its application. *Applied Soft Computing* **32**, 544–552 (2015).
26. Saurabh, P. & Verma, B. An efficient proactive artificial immune system based anomaly detection and prevention system. *Expert Systems With Applications*, 311–320. (2016)
27. Yang, T., Chen, W., Li, T., et al. An antigen space density based real-value negative selection algorithm. *Applied Soft Computing*, 860–874. (2017).
28. Sun, Z. et al. An Intrusion Detection Model for Wireless Sensor Networks with an Improved V-Detector Algorithm. *IEEE Sensors Journal* **18**(5), 1971–1984 (2018).
29. Wang, W. et al. Intrusion detection and security calculation in industrial cloud storage based on an improved dynamic immune algorithm. *Information Sciences* **501**, 543–557 (2019).
30. Yang, C. et al. Negative Selection Algorithm Based on Antigen Density Clustering. *IEEE Access* **8**, 44967–44975 (2020).
31. Xi, L. et al. Multisource neighborhood immune detector adaptive model for anomaly detection. *IEEE Transactions on Evolutionary Computation* **25**(3), 582–594 (2021).
32. Kim, Y. J., Nam, W. & Lee, J. Multiclass anomaly detection for unsupervised and semi-supervised data based on a combination of negative selection and clonal selection algorithms. *Applied Soft Computing* **122**, 108838 (2022).
33. Huang, H. et al. An artificial immunity based intrusion detection system for unknown cyberattacks. *Applied Soft Computing* **148**, 110875 (2023).
34. Liu, X. et al. A novel immune detector training method for network anomaly detection. *Applied Intelligence* **54**(2), 2009–2030 (2024).

35. Peng, L. et al. A preliminary exploration of the T cells multilayer immune tolerance model. *The Journal of Supercomputing* **81**(4), 565 (2025).
36. Guerroumi, M. & Derhab, A. NSNAD: negative selection-based network anomaly detection approach with relevant feature subset. *Neural Computing and Applications* **32**(8), 3475–3501 (2020).
37. Ji, Z. & Dasgupta, D. Revisiting Negative Selection Algorithms. *Evolutionary Computation* **15**(2), 223–251 (2007).
38. Tavallae, M., Bagheri, E., Lu, W., et al. A detailed analysis of the KDD CUP 99 data set[C]//2009 IEEE symposium on computational intelligence for security and defense applications. IEEE, 2009: 1–6.
39. Moustafa, Nour, & Jill, Slay. “UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set).” Military Communications and Information Systems Conference (MilCIS), 2015. IEEE, (2015).
40. Moustafa, Nour, & Jill, S. “The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 dataset and the comparison with the KDD99 dataset.” Information Security Journal: A Global Perspective (2016): 1–14.
41. Liu, R. et al. Artificial intelligence for fault diagnosis of rotating machinery: A review. *Mechanical Systems and Signal Processing* **108**, 33–47 (2018).
42. Chang, C.C., Lin, C.J.: LIBSVM: a library for support vector machines. ACM transactions on intelligent systems and technology (TIST), 2011, 2(3): 1–27. Version 3.24 released on September 11, 2019. Last accessed:2025-08-01. Available: <https://www.csie.ntu.edu.tw/~cjlin/libsvm/index.html>
43. MATLAB. fitctree: Fit binary decision tree for multiclass classification. Last accessed: 2025-08-01. Available: <https://www.mathworks.com/help/stats/fitctree.html>
44. MATLAB. TreeBagger class: Bag of decision trees. Last accessed: 2025-08-01. Available: <https://www.mathworks.com/help/stats/treebagger-class.html>
45. Hastie, T., Tibshirani, R. & Friedman J. The elements of statistical learning. (2009).

Acknowledgements

This work was supported by the Special Basic Cooperative Research Programs of Yunnan Provincial Undergraduate Universities' Association (grant No. 202101BA070001-148), Yunnan Provincial Department of Education Science Research Fund Project (grant No. 2023J1102), Reserve Talent Project for Young and Middle aged Academic and Technical Leaders (grant No.202405AC350069), and the National Natural Science Foundation of China (grant No.62241306).

Author contributions

Zhiyong Li: Conceptualization, Methodology, Software, Validation, Visualization, Writing. Xiang Wei: Formal analysis, Resources. Chunyan Li: Investigation. Jianhong Sun: Data curation.

Declarations

Competing interests

The authors declare no competing interests..

Additional information

Correspondence and requests for materials should be addressed to Z.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025